

An EIG-Inspired Clarification Pipeline for Code Generation

Priyangshu Mandal

September 2026

Repository:

https://github.com/Itsoofficiallymadhav/NLP_Clarification/tree/priyangshu-mandal

Contents

1	Introduction	2
2	Relationship to the Source Paper	2
3	Method	2
3.1	Dataset and task representation	2
3.2	Baseline generation	3
3.3	Candidate sampling and ambiguity detection	3
3.4	EIG-inspired clarification and selection	3
3.5	Custom Codeforces-style task	4
3.6	Evaluation	4
4	Results	4
4.1	Stored experiment results	4
4.2	Ambiguity analysis	5
4.3	Interpretation	5
4.4	Human-guided oracle evaluation	5
4.5	Codeforces-style case study	5
5	Conclusion	5

Abstract

This report documents a repository-level implementation based on *20 Questions for Code: Improving Code Generation with Information-Theoretic Clarification* [2]. The source paper proposes sampling candidate programs, selecting executable clarification tests by expected information gain (EIG), querying an oracle, and updating candidate beliefs before submitting code. This project implements a smaller MBPP-focused version of that idea: it samples eight candidates, extracts disagreement probes from existing MBPP tests, supports up to four clarification questions, and includes both an automated and a human-guided oracle path. The strongest stored baseline run passes 42 of 50 tasks (84.0%), while the strongest stored clarification run passes 20 of 21 tasks (95.2%). These runs are not paired experiments, so the comparison is descriptive rather than a causal estimate.

1 Introduction

Automatic code generation systems can produce syntactically valid programs while still misunderstanding an underspecified requirement. The supplied source paper, *20 Questions for Code*, studies this problem by treating clarification as an information-theoretic question-selection problem: sample plausible programs, ask executable yes/no tests that distinguish them, and use the answers to select a better program [2].

This repository is a focused implementation and exploration of that idea. It is not a reproduction of every experiment in the source paper. The paper reports automated EIG experiments on HumanEval+ and MBPP+ with larger candidate and test pools, while this project uses the standard MBPP dataset, existing assertion-derived probes, and a human-guided oracle evaluation mode. The report therefore evaluates the implemented pipeline and labels these differences explicitly.

The experiments use the Mostly Basic Python Problems (MBPP) benchmark, which contains natural-language programming tasks paired with reference tests [1]. The central question is:

Can the source paper’s EIG-based clarification idea be implemented over MBPP candidates, and can answers to focused output questions improve the final program selected for evaluation?

The project is organized into four phases:

1. establish a one-shot code-generation baseline;
2. sample multiple candidates and measure behavioral ambiguity;
3. use an automated clarification loop to update candidate weights;
4. evaluate a human-guided oracle version of the clarification loop.

The report summarizes the implemented pipeline and the experiment artifacts saved in the repository. The human-guided oracle mode is treated as a complete evaluation setting in which a person answers targeted binary questions, the candidate distribution is updated, and the selected program is checked against the task tests.

2 Relationship to the Source Paper

The source paper’s central object is a posterior distribution over candidate programs. It generates candidate clarification tests, scores them by expected information gain and test quality, queries a reference implementation, and updates candidate weights with a soft Bayesian rule. The repository follows the same conceptual loop but narrows the implementation in several ways:

The repository should therefore be described as an EIG-inspired implementation and exploratory reproduction, not as an independent implementation of all source-paper experiments.

3 Method

3.1 Dataset and task representation

Each MBPP example contains a problem description, a list of tests, optional setup code, and a task identifier [1]. A generated response is first reduced to Python source by extracting a fenced code block when present. The evaluator then executes the optional setup code, the generated candidate, and each assertion in a shared namespace.

Table 1: Scope differences between the source paper and this repository.

Aspect	Source paper	This repository
Benchmarks	HumanEval+ and MBPP+	Standard MBPP task splits
Candidate generation	16 candidates	8 candidates
Clarification tests	Model-generated test pool	Existing assertion-derived disagreement probes
Oracle	Reference implementation	Automated oracle and human-guided oracle
Question budget	Up to 4 in the reported configuration	Up to 4
Evaluation	pass@1 with broader test suites and cost analysis	Assertion execution with stored pass/error records

3.2 Baseline generation

The main generation implementation uses `Qwen2.5-Coder-7B-Instruct` with 4-bit quantization. The prompt asks for a single Python code block and may include the first MBPP test as an additional behavioral constraint. The recorded baseline artifacts include runs of 20, 30, and 50 tasks. Because these files were created at different stages and with different selections, they are reported separately.

3.3 Candidate sampling and ambiguity detection

For each task in the clarification pipeline, the system samples eight candidate programs at a nonzero temperature. Invalid Python is discarded, and candidates are deduplicated by their outputs on literal probe inputs extracted from the MBPP assertions. A probe input is considered informative when valid candidates produce more than one output group. This makes ambiguity operational: it is disagreement among executable candidates, not merely uncertainty in the model’s text. Unlike the source paper, the current repository does not generate a separate pool of clarification tests from scratch; it derives questions from observed candidate disagreement.

3.4 EIG-inspired clarification and selection

For a candidate question t , each candidate program either agrees with the hypothesized output, disagrees, or produces an undefined/error outcome. Let p_t be the weighted fraction of defined candidates that agree with the hypothesized output. The implemented base score follows the binary-entropy form used by the source paper:

$$\text{EIG}(t) = H_b(\epsilon + (1 - 2\epsilon)p_t) - H_b(\epsilon),$$

where H_b is binary entropy and $\epsilon = 0.02$ models oracle noise. The repository then multiplies this quantity by defined-outcome weight, a balance bonus, and a penalty for undefined outcomes. With weights w_j , answers update softly: candidates consistent with the answer receive factor $1 - \epsilon$, inconsistent candidates receive factor ϵ , and undefined candidates receive factor $\eta = 0.3$, followed by normalization. The ask-or-submit rule compares the expected post-question maximum weight against the current maximum using $\gamma = 0.85$.

The highest-scoring disagreement question is asked for at most four rounds. The automated oracle setting answers by executing the question against reference code when available. The human-guided oracle setting presents the same binary question to a person, records the answer, applies the

same belief update, and evaluates the highest-weight candidate against the task’s test list. These are two complete oracle modes for the same clarification loop.

3.5 Custom Codeforces-style task

To test whether the clarification workflow extends beyond MBPP-shaped prompts, the human-guided oracle notebook includes a Codeforces-style custom task, identified as `cf_1097b`. The task asks whether a list of rotation angles can be assigned clockwise or counterclockwise directions so that the signed total is a multiple of 360 degrees. The workflow generates eight candidate programs, measures disagreement using three public examples, asks up to four targeted binary questions when candidates diverge, and selects the highest-weight candidate. The selected program is then checked with additional cases, including a single 180-degree rotation, which should return false, and four 90-degree rotations, which should return true.

3.6 Evaluation

For each task, a result is recorded as a Boolean pass flag and an error string. A pass means that all provided assertions execute successfully. Runtime errors and assertion failures are retained for diagnosis. This is a functional correctness measure, not a measure of style, runtime efficiency, token cost, confidence calibration, or semantic similarity to the reference implementation. Consequently, this repository does not reproduce the source paper’s full pass@1, token-efficiency, confidence-interval, or ablation claims.

4 Results

4.1 Stored experiment results

Table 2 summarizes the JSON artifacts currently stored under `results/`. The runs are intentionally shown separately because they do not all use the same task count or task selection.

Table 2: Pass rates in the stored result artifacts.			
Run	Tasks	Passed	Rate
Baseline, 20-task artifact	100	51	51.0%
Baseline, 7B, 30 tasks	30	20	66.7%
Baseline, 7B, 50 new tasks	50	33	66.0%
Baseline, 7B, 50 random tasks	50	42	84.0%
Clarification, 15 ambiguous tasks	15	8	53.3%
Clarification, 25-task run	25	20	80.0%
Clarification, 21-task random run	21	20	95.2%

The strongest stored clarification run has the highest observed pass rate, but it is based on 21 tasks and is not paired with the 50-task random baseline. Therefore, the result supports the feasibility of this EIG-inspired implementation, but it does not reproduce the source paper’s benchmark claims or establish a controlled improvement attributable to clarification.

4.2 Ambiguity analysis

The phase-two artifacts show that several tasks have multiple executable behaviors among the sampled candidates. For example, task 603 produces six unique candidate behaviors and three disagreement points in the saved analysis. This is the type of case targeted by the source paper’s method: multiple plausible interpretations can be exposed by asking about a concrete input and output. In this repository, those questions are drawn from existing MBPP assertions rather than generated as a standalone test-generation stage.

4.3 Interpretation

The stored results are useful as engineering evidence that the candidate-disagreement and weight-update loop runs end to end. They are not a direct comparison to the source paper’s reported MBPP+ values. The task counts, task selections, candidate-generation settings, and evaluation suites differ, and the clarification artifacts are not consistently paired with a same-task one-shot baseline.

4.4 Human-guided oracle evaluation

The human-guided oracle setting provides a complete interactive evaluation workflow. It loads an ambiguous MBPP task and its candidate programs, selects a high-information input/output question, presents the binary clarification question to a person, updates the candidate weights from the answer, and evaluates the final selected program against the task tests. Each interaction records the task identifier, question inputs, hypothesized outputs, answers, scores, final candidate, final weights, and evaluation outcome. This setting makes the human oracle’s role explicit while preserving the same candidate-ranking and EIG-inspired update procedure as the automated setting.

4.5 Codeforces-style case study

The notebook also applies the complete human-guided oracle workflow to the custom task `cf_1097b`. The public examples include angle lists such as `[10, 20, 30]`, `[10, 10, 10]`, and `[120, 120, 120]`, with expected outcomes that distinguish valid signed-rotation combinations from invalid ones. After clarification, the selected candidate is evaluated on two additional checks: `[180]` must return false because neither sign reaches 0 modulo 360, while `[90, 90, 90, 90]` must return true because four equal rotations sum to 360. This case study demonstrates that the clarification loop is not tied to the MBPP dataset format; it can also operate on a custom algorithmic prompt with user-defined public and hidden tests.

5 Conclusion

This repository implements a focused version of the information-theoretic clarification idea introduced in *20 Questions for Code* [2]. It covers the essential loop: candidate sampling, executable behavioral disagreement, EIG-inspired question scoring, soft belief updates, and final code evaluation. The stored results show that this loop can run end to end and select passing solutions on many tasks.

The implementation remains narrower than the source paper. It uses standard MBPP rather than MBPP+, derives questions from existing tests rather than generating a full candidate test pool, and provides both automated and human-guided oracle evaluation modes. Its stored runs are exploratory and should not be presented as reproductions of the source paper’s headline results.

The custom Codeforces-style rotation task further demonstrates the intended scope of the workflow. By supplying its own public examples and additional hidden checks, it shows how candidate disagreement and human-guided clarification can be applied to an algorithmic problem outside the MBPP dataset.

The next experiment should use a fixed task set and paired comparisons. For every task, run a one-shot baseline and the clarification method from the same candidate pool, record the number of questions, and report pass-rate differences with uncertainty across tasks. To move closer to the source paper, the project can add model-generated candidate tests, random-test and self-consistency baselines, token accounting, and confidence intervals while retaining the human-guided oracle mode as a complementary evaluation setting.

Finally, the evaluator should remain isolated from the model-generation code and should preserve setup code, assertion failures, and runtime errors. This makes failures diagnosable and prevents improvements in reporting from being confused with improvements in generated-program correctness.

References

- [1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [2] Ria Garg, Alexandra Kim, and Julia Xi. 20 questions for code: Improving code generation with information-theoretic clarification. Stanford CS224N Custom Project, 2026.